



Register

SDK ▾

FAQ

Changelog

Pricing plans ▾



Search



SDK update rules

▾ Android

Changelog

Important update for outdated versions of the AppMetrica SDK

List of modules

Excluding geo from the list of dependencies

Disabling automatic collection of Ad Revenue data

Getting advertising IDs

> Migration guides

▾ AppMetrica SDK

SDK integration

Library features

Tracking user activity

Logging

Usage examples

Analyzing crashes

AppMetrica Gradle Plugin

Deferred deeplinks support

Product flow and offer events

Error descriptions

SDK reference

> Push SDK

> iOS

> Unity

> Flutter

> React Native

> Push campaigns

> Setting up data collection

SDK Developer docs / [Android](#) / AppMetrica SDK / AppMetrica Gradle Plugin

AppMetrica Gradle Plugin

AppMetrica lets you collect information about native and Java crashes. You can analyze them in the [Crashes](#) report. See [Crashes/errors](#).

To reduce the size of an app, [optimize](#) the code during a release build.

If the code was compressed and obfuscated during the build of an Android application, information about crashes is transferred in obfuscated form. To extract data for analysis from such crash logs, AppMetrica performs server-side deobfuscation.

To do this, upload the mapping files or debug symbols of SO files to AppMetrica, either automatically when building the app, or manually via the web interface.

To send files, enable the AppMetrica Gradle Plugin.

Restrictions

- The plugin is applied only to modules with `com.android.application`. It is not applied to `com.android.library`.
- Upload mapping files if you use [ProGuard](#) or [R8](#). If you don't compress or obfuscate the code, don't enable the plugin.
- The plugin's operation depends on the versions of `com.android.tools.build:gradle` (AGP) and gradle. The plugin is compatible with the following versions:
 - `com.android.tools.build:gradle` from [7.2.0](#) to [8.13.+](#) (except for [8.0.+](#));
 - gradle from [7.4](#) to [9.2.1](#). We don't guarantee that the plugin will work with other versions.

The plugin's functionality is not guaranteed when using Gradle [8.0](#) and AGP [7.4.*](#) together.

- For local builds, it is recommended to disable the plugin — this speeds up the build. Enable it only on CI. A convenient way is to control the `enable` flag through an environment variable:

```
appmetrica {
    enable.set(providers.environmentVariable("CI").map { it == "true" }.orElse(false))
}
```

Connecting the plugin

To enable the plugin:

- Add the following dependency to the Gradle root file:

```
plugins {
    id("io.appmetrica.analytics") version "2.1.0" apply false
}
```

Enabling plugin using [buildscript](#)

- Add the plugin and a minimal configuration to the app's Gradle file:

```
plugins {
    id("com.android.application")
    id("io.appmetrica.analytics")
}

appmetrica {
    postApiKey.set("Post Api key")
    ndk {
        enable.set(true) // Optional. Enable if your project has native code and you need it
    }
}
```

The rest of the parameters and their default values are described in [Plugin parameters](#).

Common tasks

In the typical case, you don't need to change any settings: `enable` defaults to `true` only for `buildType = 'release'` `ndk.enable` defaults to `false` (enable it only if your project has native code and you need native crashes). So for a pr obfuscation outside of `release` and without NDK symbols, the plugin already works optimally out of the box.

Collect mapping and NDK symbols for a separate non-release buildType without spending the upload quota

Plugin parameters

Example configuration with all available parameters:

```
appmetrica {
    postApiKey.set("Post Api key")
    enable.set(true) // Optional
    offline.set(false) // Optional
    mappingFile.set(file("path/to/mapping.txt")) // Optional
    enableAnalytics.set(true) // Optional
    allowTwoAppMetricas.set(false) // Optional
    ndk {
        enable.set(false) // Optional
        soFiles.from(file("path/to/so")) // Optional
        additionalSoFiles.from(file("path/to/extra")) // Optional
        addNdkCrashesDependency.set(true) // Optional
    }
}
```

Parameter	Description
<code>postApiKey*</code>	Post API key. <code>String</code> . You can get the Post API key in the Settings section in AppMetrica. It's use identify your app. If <code>offline = true</code> , the parameter is optional. Note on hierarchical configuration: the value must match across all sources is set. If different sources specify different values, the plugin logs an error ar the key as not set (see Hierarchical configuration).
<code>enable</code>	Enables or disables the plugin for the given build variant. <code>Boolean</code> . If the plugin is disabled, the apk is not changed, mapping is neither collecte uploaded, and resource generation is not performed. Doesn't affect collecting or uploading NDK symbols — that's controlled by t independent <code>ndk.enable</code> parameter (see below). By default, <code>true</code> only for <code>buildType = 'release'</code> , otherwise <code>false</code> . Supports hierarchical configuration (see Hierarchical configuration).
<code>offline</code>	Enables the <code>offline</code> mode. <code>Boolean</code> . Affects the upload of both mapping and NDK symbols at the same time (reg whether <code>enable</code> or <code>ndk.enable</code> is on — offline mode is shared between I Acceptable values: <code>true</code> — Doesn't upload either mapping or symbols to AppMetrica. An placed in <code>build/appmetrica/<variant>/result/</code> , paths are writte log. You can upload the archives manually via the web interface. <code>false</code> — Automatically uploads mapping and symbols. The default value is <code>false</code> . Supports hierarchical configuration (see Hierarchical configuration).
<code>mappingFile</code>	Path to the mapping file. <code>RegularFileProperty</code> (in Kotlin DSL set via <code>file("...")</code>).

	The default value is the mapping file from the build (the <code>OBFUSCATION_MAPPING_FILE</code> AGP artifact). Override it only if you want to mapping from a non-standard location. Supports hierarchical configuration (see Hierarchical configuration).
<code>enableAnalytics</code>	Lets plugin usage statistics be sent to AppMetrica. <code>Boolean</code>. Acceptable values: <code>true</code> — Sending statistics is enabled. <code>false</code> — Sending statistics is disabled. The default value is <code>true</code>.
<code>allowTwoAppMetricas</code>	Behavior when two different AppMetrica libraries (<code>io.appmetrica.analytics:analytics</code> and <code>com.yandex.android:mobmetricalib</code>) are present in the project at the same time. <code>Boolean</code>. Acceptable values: <code>false</code> — If both libraries are found, the build fails with an error. <code>true</code> — The presence of both libraries is allowed, a warning is written log. The default value is <code>false</code>. Supports hierarchical configuration (see Hierarchical configuration).
<code>ndk</code>	This parameter is required to load symbols from the SO file to track native crashes.
<code>ndk.enable</code>	Enables or disables processing and uploading symbols from SO files. <code>Boolean</code>. If enabled, symbol processing and upload tasks run automatically after the build. The default value is <code>false</code>. Supports hierarchical configuration (see Hierarchical configuration).
<code>ndk.soFiles</code>	SO files to process. <code>ConfigurableFileCollection</code>. Use <code>.from(...)</code> to add files. By default, the plugin takes SO files from the <code>MERGED_NATIVE_LIBS</code> AGP artifact, and this already includes <code>src/main/jniLibs/**</code> and the output of CMake/ndk-build. Override it if your SO files are in a non-standard location or the plugin can't find them. Hierarchical configuration behavior: collections from all sources are merged (see Hierarchical configuration).
<code>ndk.additionalSoFiles</code>	Additional SO files. <code>ConfigurableFileCollection</code>. Use <code>.from(...)</code> to add files. Unlike <code>soFiles</code>, it doesn't replace the default but extends it. Hierarchical configuration behavior: collections from all sources are merged (see Hierarchical configuration).
<code>ndk.addNdkCrashesDependency</code>	Automatically add the <code>io.appmetrica.analytics:analytics-ndk-crashes</code> dependency to the <code><variant>Implementation</code> configuration. <code>Boolean</code>. The default value is <code>true</code>. Supports hierarchical configuration (see Hierarchical configuration).

**Note**

If the `ndk.enable` parameter is enabled, symbol processing and upload tasks run automatically after the build. If additional manual configuration is required.

Hierarchical configuration

The plugin supports four configuration levels:

```
// For Kotlin DSL, import the matching extension function for your AGP version:
// import io.appmetrica.analytics.gradle.agp8.appmetrica // for AGP 8.x
// import io.appmetrica.analytics.gradle.agp7.appmetrica // for AGP 7.x

appmetrica {
    // Global configuration
    variants {
        create("<variantName>") {
            // Variant configuration
        }
    }
}

android {
    buildTypes {
        named("<buildTypeName>") {
            appmetrica {
                // buildType configuration
            }
        }
    }
    productFlavors {
        named("<flavorName>") {
            appmetrica {
                // flavor configuration
            }
        }
    }
}
```



Note

For each build variant, the plugin collects values from all levels and applies a resolution strategy to them — each parameter has its own. **This is not the "usual" priority where "the top level wins" — the behavior depends on parameter type.**

`enable`, `offline`, `ndk.enable`, `allowTwoAppMetricas`, `enableAnalytics`, `ndk.addNdkCrashesDependency`

Logic: "disabling always wins".

- If at least one source has the value `false` — the result is `false`.
- Otherwise, if at least one source has the value `true` — the result is `true`.
- Otherwise, the default value is used.

Example:

```
// For Kotlin DSL, import the matching extension function for your AGP version:
// import io.appmetrica.analytics.gradle.agp8.appmetrica // for AGP 8.x
// import io.appmetrica.analytics.gradle.agp7.appmetrica // for AGP 7.x

appmetrica {
    enable.set(true)
}

android {
    buildTypes {
        debug {
            appmetrica {
                enable.set(false)
            }
        }
    }
}
```

As a result, `enable` is `false` for debug and `true` for all other variants.

`postApiKey`

Logic: "the value must be unique".

- If the parameter is set in only one source — that value is used.
- If it is set in several sources with the same value — that value is used.
- If different sources specify different values — the error `Conflicting values for postApiKey` is logged, and the value (an empty string) is used. Upload then won't happen — the plugin additionally emits the warning `postApiKey set`.

Therefore, set `postApiKey` **only in one place** — usually in the global block or in the relevant flavor.

`mappingFile`

Logic: "priority lookup".

Source priority: global block > per-variant > per-buildType > the first configured flavor > default value (mapping from the value from the first source where the parameter is set is used).

`ndk.soFiles`, `ndk.additionalSoFiles`

Logic: "collection merging".

All non-empty `.from(...)` from all levels are **merged** into a single collection. If the collection is not configured at any level, `soFiles` falls back to the AGP default, and `additionalSoFiles` falls back to an empty collection. To replace the default files, set `soFiles.from(...)` in one of the sources — the default is then not used.

Example

```
// For Kotlin DSL, import the matching extension function for your AGP version:
// import io.appmetrica.analytics.gradle.agp8.appmetrica // for AGP 8.x
// import io.appmetrica.analytics.gradle.agp7.appmetrica // for AGP 7.x

// Global settings for all variants
appmetrica {
    enableAnalytics.set(true)
}

android {
    buildTypes {
        release {
            // NDK symbol upload is needed only for release builds
            appmetrica {
                ndk {
                    enable.set(true)
                }
            }
        }
    }
}

productFlavors {
    // Each flavor has its own key — for example, different apps for prod and dev
    create("prod") {
        appmetrica {
            postApiKey.set("prod-post-api-key")
        }
    }
    create("dev") {
        appmetrica {
            postApiKey.set("dev-post-api-key")
        }
    }
}

// Per-variant: targeted override for a specific variant
appmetrica {
    variants.create("prodRelease") {
        ndk {
            additionalSoFiles.from(file("extra-symbols"))
        }
    }
}
```



Tip

To find out which values the plugin actually applied to a variant, look at the build log: for each parameter, a line in the form `Using value 'X' from <source> for <parameter>` or `Using default value '...' for`

Manual loading

For manual upload, set `offline = true` mode. This is needed so that the plugin generates `info.txt` with meta info which the server will associate the archive with the build.

- 1. In the `app/build.gradle` file, turn on `offline` mode:

```
appmetrica {
    offline.set(true)
}
```

In `offline` mode, the `postApiKey` parameter is optional.

- 2. Build the desired variant of the application (for example, `./gradlew assembleRelease`). Archives will appear in `<appModule>/build/appmetrica/<variant>/result/`:
 - `mapping.zip` — for Java crashes;
 - `symbols.zip` — for native crashes, if `ndk.enable` is enabled.
- 3. In the AppMetrica interface, go to the app settings from the menu on the left.
- 4. Go to the **Crashes** → **Android** tab.
- 5. Click **Choose file** and upload the desired archive.

Plugin tasks

For each build variant `<variant>` (`Release`, `ProdRelease`, etc.), the plugin registers the following tasks:

Task name	Purpose
<code>check<variant>AppMetricaDependencies</code>	Checks that exactly one AppMetrica library is used in the project (<code>allowTwoAppMetricas</code>).
<code>create<variant>AppMetricaRes</code>	Generates resources in <code>build/appmetrica/<variant>/res/</code> ; result is merged into the application resources.
<code>zip<variant>AppMetricaFiles</code>	Builds <code>mapping.zip</code> (<code>info.txt</code> + mapping).
<code>upload<variant>AppMetricaMapping</code>	Uploads <code>mapping.zip</code> to AppMetrica (in <code>offline</code> mode — the archive remains in <code>result/</code>).
<code>generate<variant>AppMetricaNdkSymbols</code>	Converts SO files to <code>.ysym</code> .
<code>zip<variant>AppMetricaNdkFiles</code>	Builds <code>symbols.zip</code> (<code>info.txt</code> + <code>.ysym</code>).
<code>upload<variant>AppMetricaNdkSymbols</code>	Uploads <code>symbols.zip</code> to AppMetrica.

Upload tasks are wired as `finalizedBy` on `assemble<variant>` and `bundle<variant>`. Therefore, a regular build (`assembleRelease` or `./gradlew bundleRelease`) automatically triggers the upload. You can run the upload manually:

```
./gradlew uploadReleaseAppMetricaMapping
./gradlew uploadReleaseAppMetricaNdkSymbols
```

Build errors and warnings

Errors that interrupt the build

- `IllegalStateException` — Code obfuscation is not enabled. Enable ProGuard or R8.

- [HttpResponseException](#) — The archive could not be uploaded. Check your internet connection and the correct `postApiKey`.
- Dependency check error — Two AppMetrica libraries (`io.appmetrica.analytics:analytics` and `com.yandex.android:mobmetricalib`) are found in the project at the same time. Remove one of them or, temporarily, `allowTwoAppMetricas = true`.

Warnings (the build does not fail, but upload may not happen)

These messages are important to monitor, especially on CI — they mean that something went wrong, but Gradle won't r

- `AppMetrica plugin is enabled for variant '<variant>' but postApiKey is not set. Upload will postApiKey or enable offline mode.` — Upload is enabled for the variant, but the key is empty. Set `postApiKey` or `enable offline`.
- `Conflicting values for postApiKey: ...` — `postApiKey` is set in different sources with different values. If one source uses an empty key — the upload will fail. Set the key in only one place or synchronize the values.
- `Using default value '...' for <parameter>` — The parameter is not set in any source, the default is used. Debugging if it seems that a setting "did not apply".

If you encounter other errors, please contact [technical support](#).

Learn more

- [Crashes and errors report](#)
- [Android documentation](#)

If you didn't find the answer you were looking for, you can use the feedback form to submit your question. Please describe the problem in as much detail as possible. Attach a screenshot if possible.

Contact support

Suggest an improvement for documentation

Was the article helpful?

☐ Yes

☐ No

[Previous](#)

[← Analyzing crashes](#)

[Deferred deeplinks s](#)